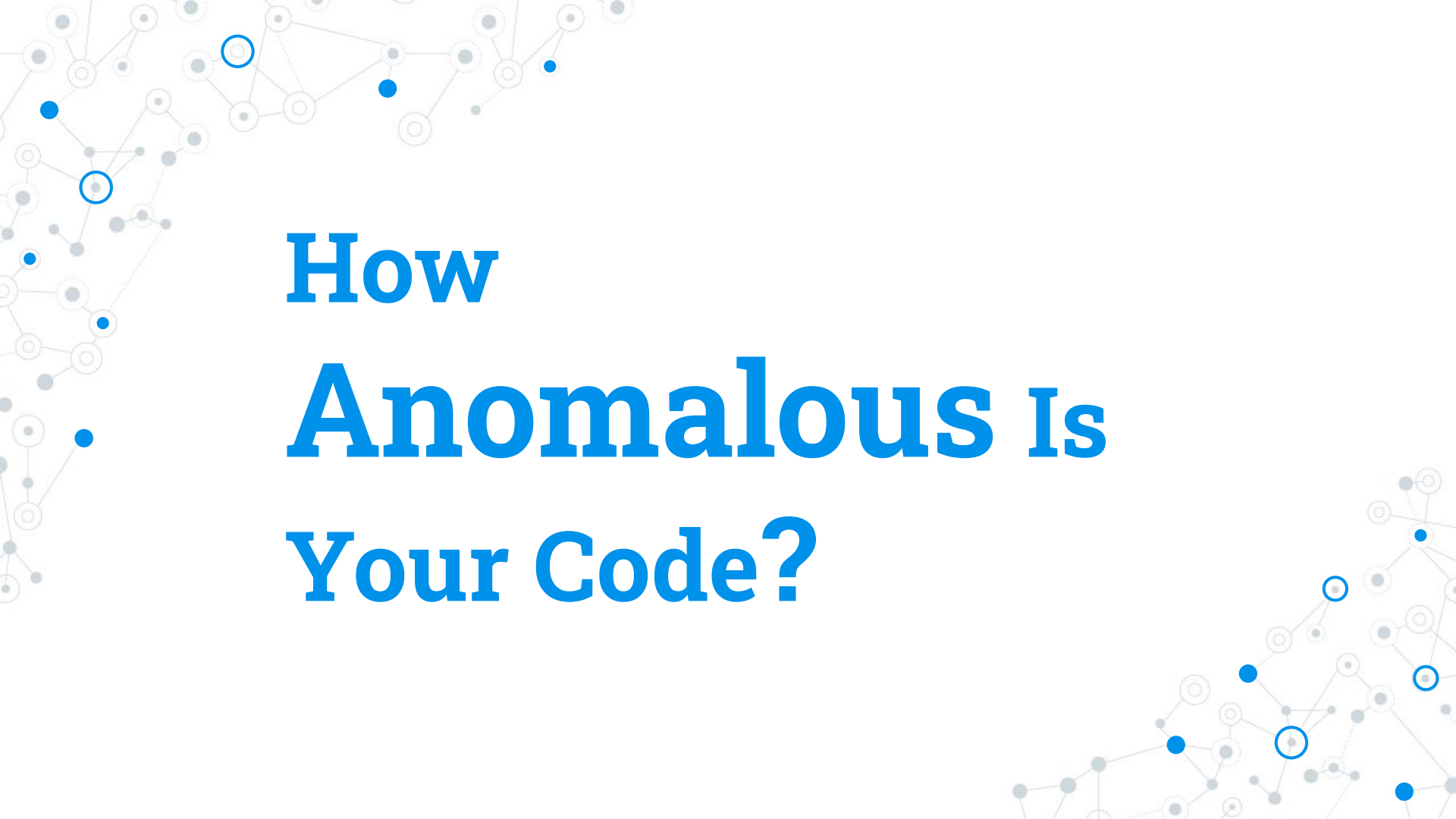


A decorative background featuring a network diagram with nodes and connecting lines. Some nodes are highlighted with blue circles or dots, and others are solid blue dots. The network is composed of various sized circles connected by thin lines, creating a complex web-like structure.

How Anomalous Is Your Code?

Hello 🖐️

I am Johannes Troppacher

Senior Software Engineer &
Team Captain

at  dynatrace



<https://github.com/JohT>

Hello 🖐️

I am Johannes Troppacher

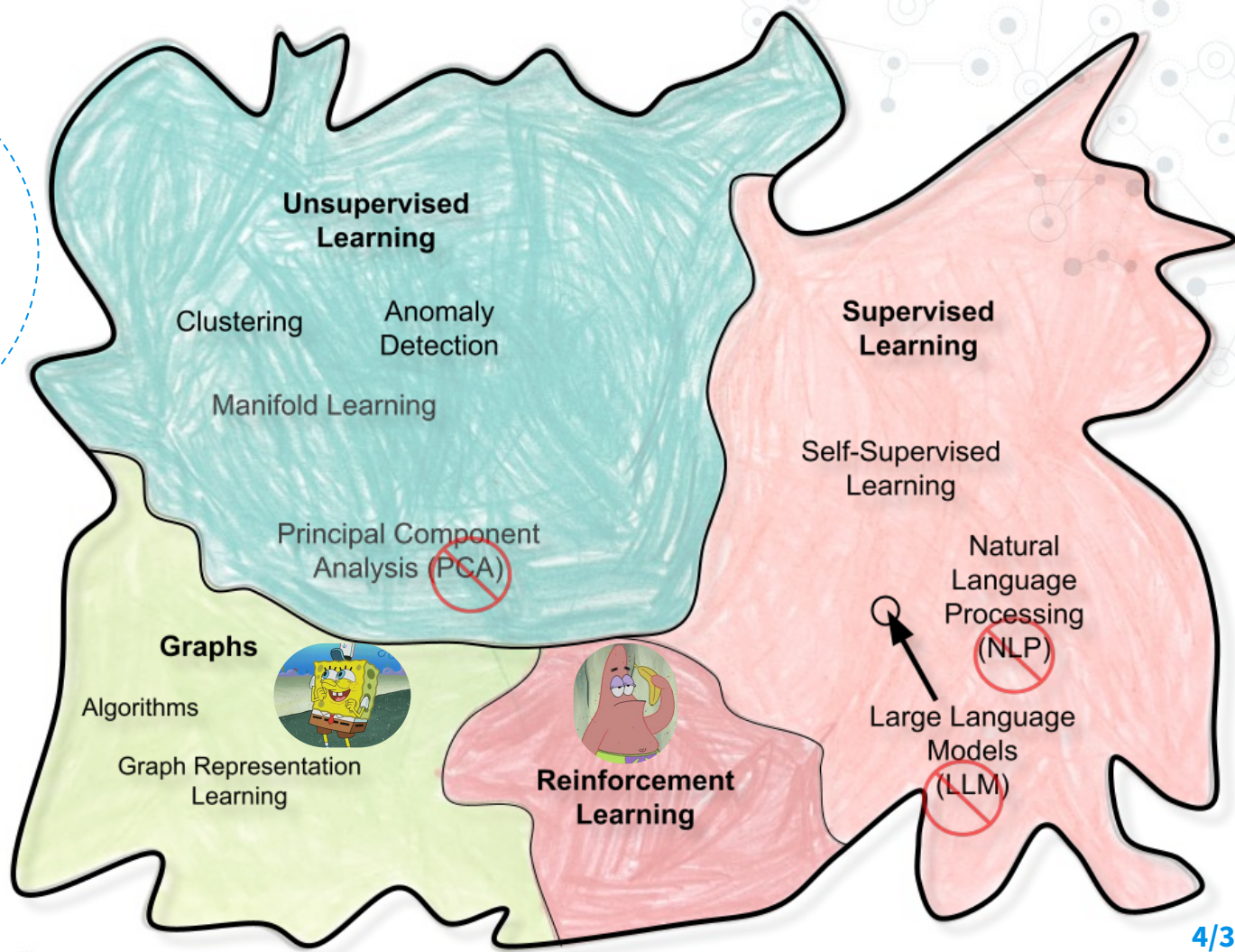
Experience:

- 🕒 25+ years
- 🕒 ...



<https://github.com/JohT>

**My highly
opinionated
view**



Why am I giving this talk?



Encourage

Look behind the curtain.

You don't need to train billion-parameter models.

More than text and image generation.



Share

Problems that I've faced.

Approaches that I've tried.

Solutions that I've found.



"Bore"

Take a break from hype.

Pause the attention war.

A contrast to AI buzzwords.

Our journey

(1) Code Graph
Database



(3) Anomaly
Detection



Demo



(2) Prepare
Machine Learning



Outlook



A decorative network diagram at the top of the slide, featuring a complex web of interconnected nodes and lines. The nodes are represented by circles of varying sizes, some solid and some dashed, connected by thin lines. The overall structure is a dense, interconnected mesh.

1) Code Database

A large, blue question mark icon centered within a dashed circular border, which is itself connected to the network diagram above it.

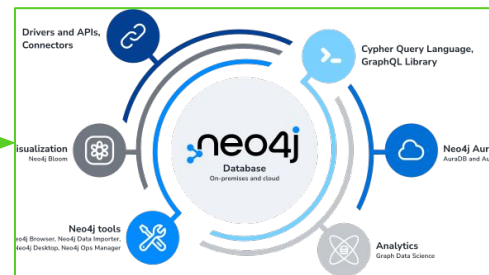
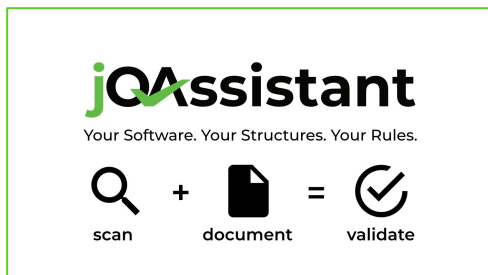
?

How can we get our **code**
into a **database**?

1) Code Database



Use *jQAssistant* to analyze **code** and *Neo4j* as a Graph **Database**

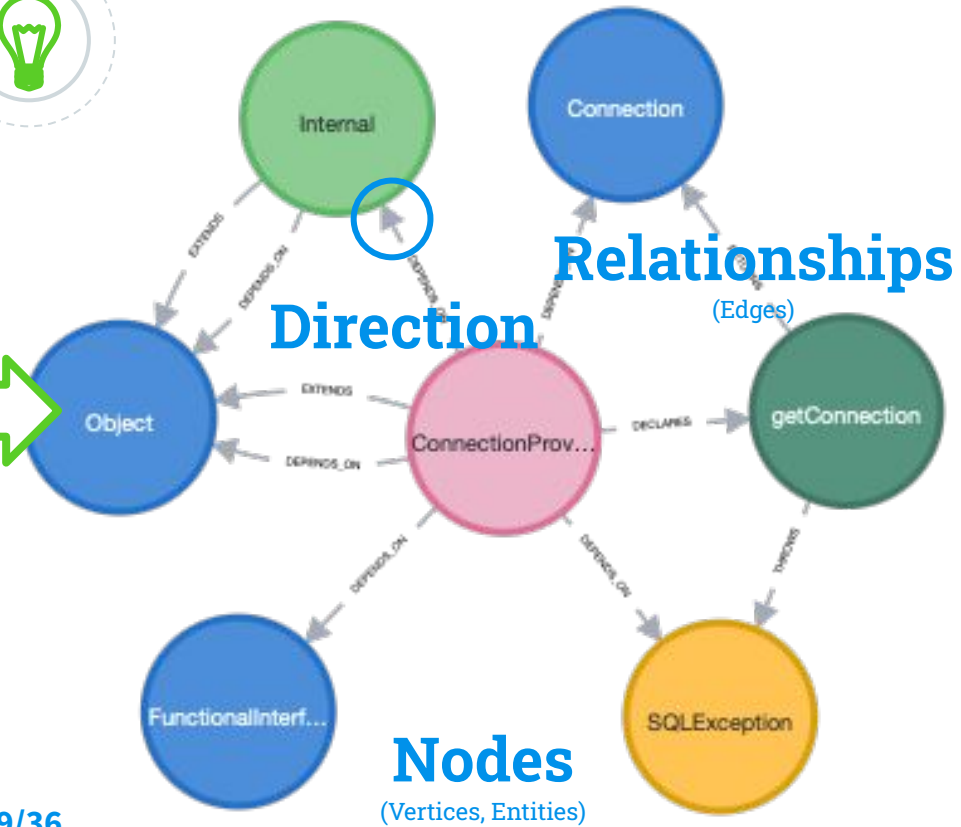


1) Code Database - Graphs



```
@Internal
@FunctionalInterface
public interface ConnectionProvider {

    /**
     * Returns a connection, ready for use.
     *
     * @return a new connection to use
     *
     * @throws SQLException when an error occurs obtaining a connection
     */
    Connection getConnection() throws SQLException;
}
```

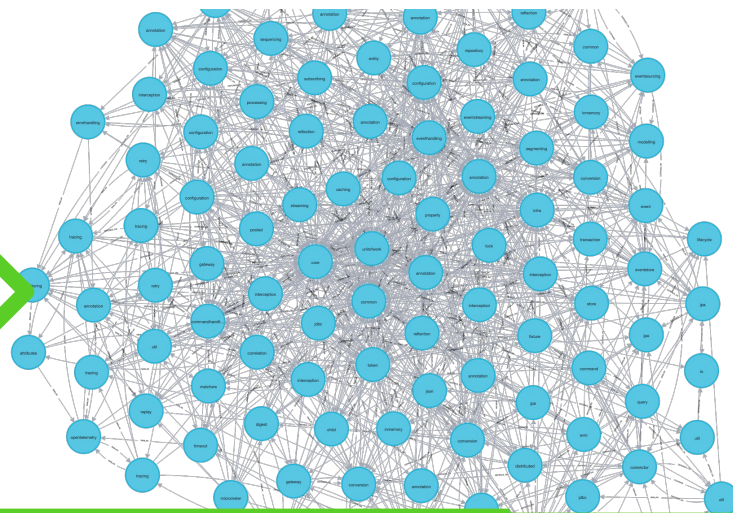
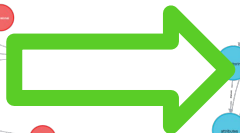
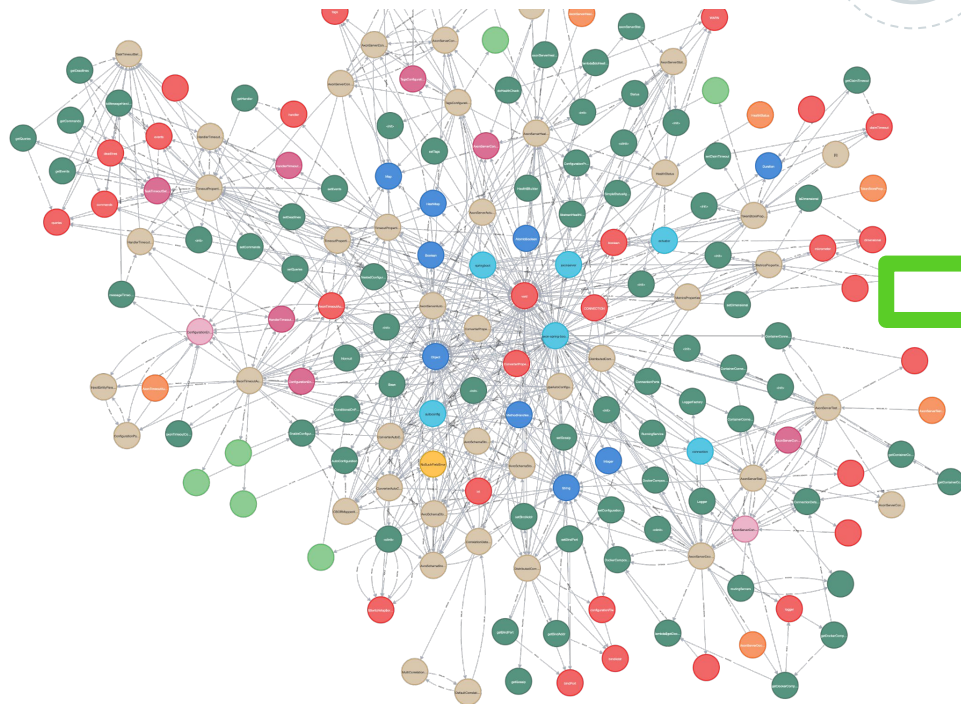


1) Code Database - Graphs

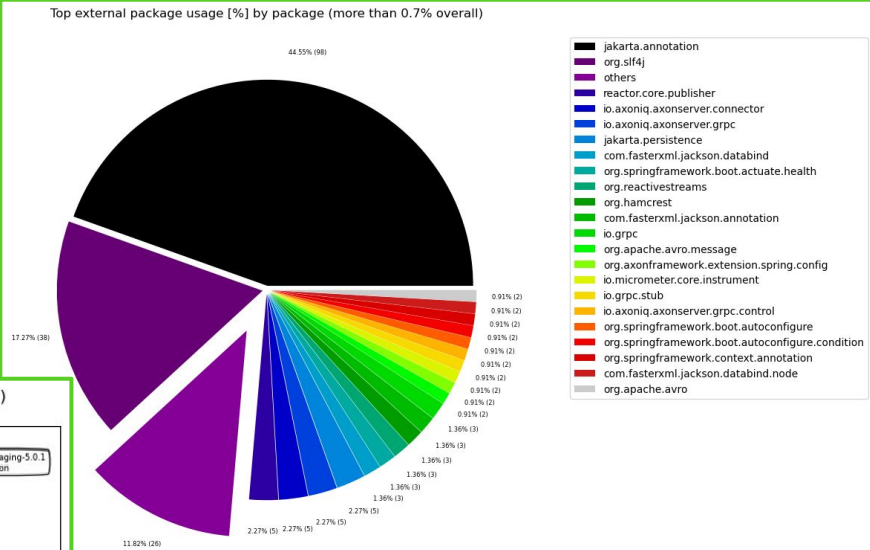
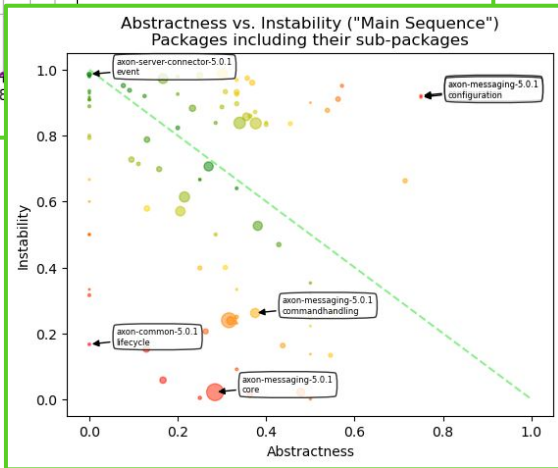
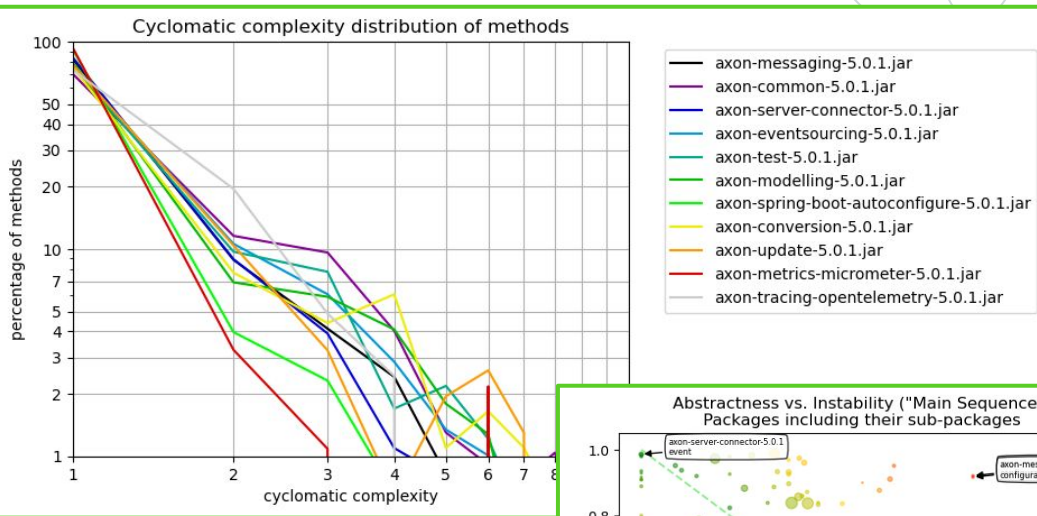
heterogeneous



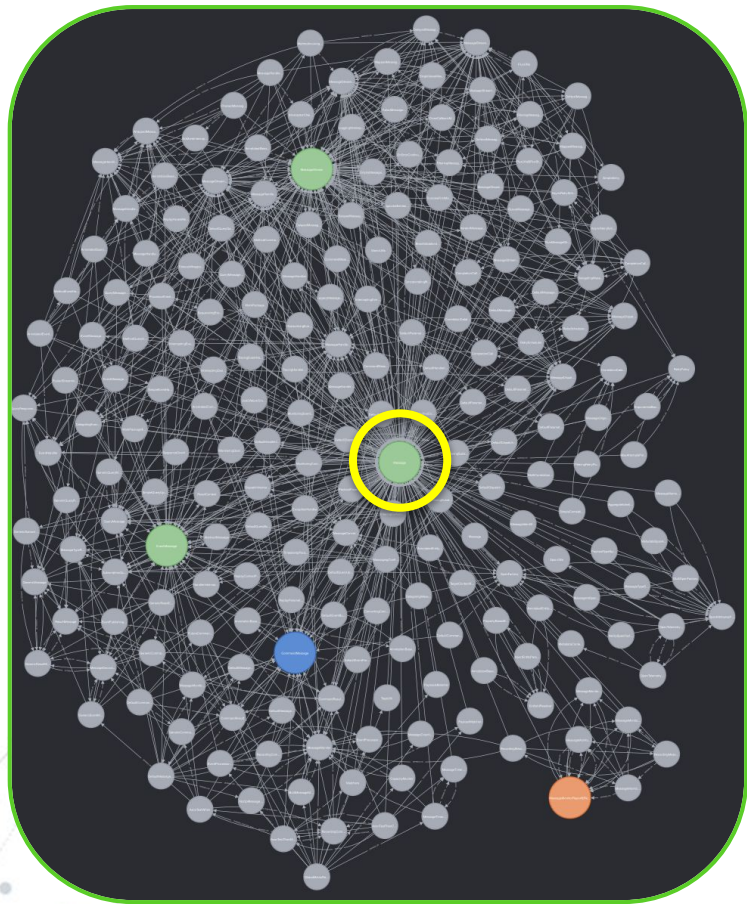
homogeneous



1) Code Database - Query Results



1) Code Database - Outstanding Examples (1/2)



The Bottleneck

Social network analogy:

- Celebrity

Graph Query:

- High betweenness centrality.
- High in-degree (incoming dependencies)

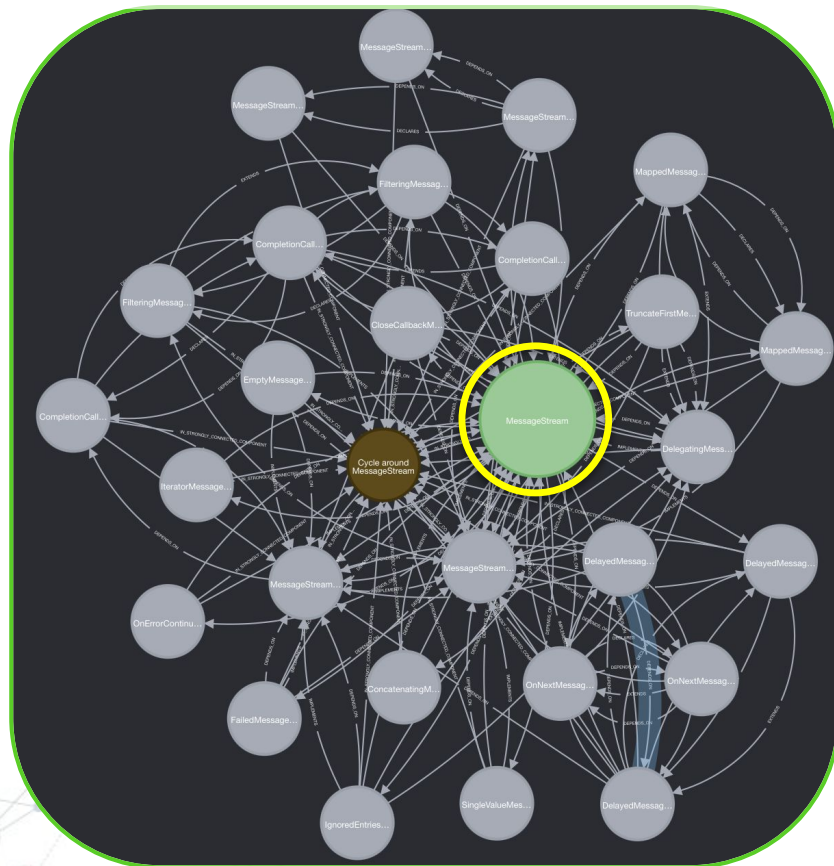
Software Architecture:

- Changes may affect many others.
- Possible “concentration of control”

Analysis:

- **Message** is THE central concept of the analyzed Open Source event-driven **AxonFramework**.

1) Code Database - Outstanding Examples (2/2)



The Hub

Social network analogy:

- Multi community influencer

Graph Query:

- High degree
- Low clustering coefficient
- **Largest cycle** (strongly connected component)

Software Architecture:

- Possible “god classes”.
- Increases coupling.
- Point of Failure.

Analysis:

- **MessageStream** central like **Message**.

2) Machine Learning Preparation



Why **Machine Learning**? What does it add?

Discovery of recurring
structural patterns
**without predefined
rules**

Identification of **unusual**
or **unexpected**
structures

Continuous **similarity**
instead of binary
decisions.

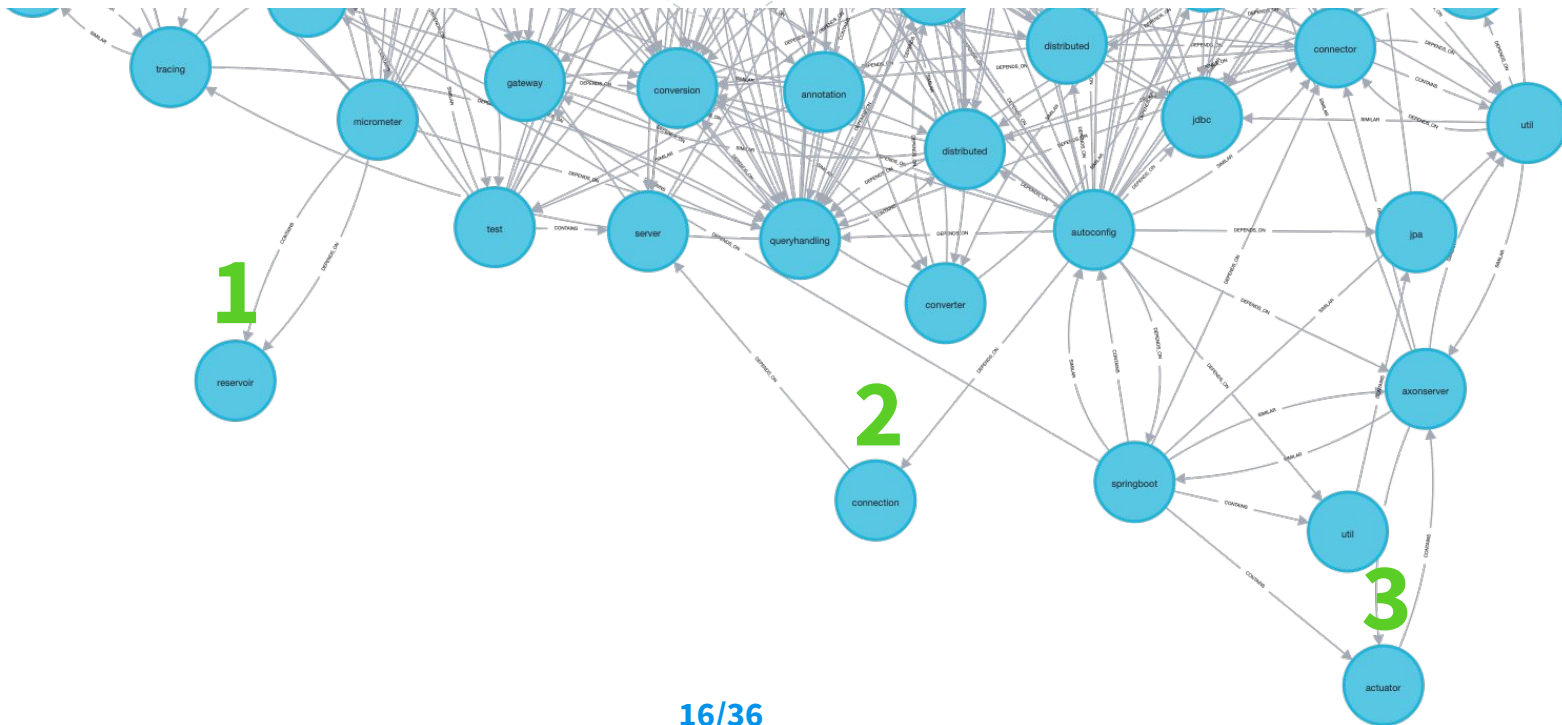
2) Machine Learning Preparation



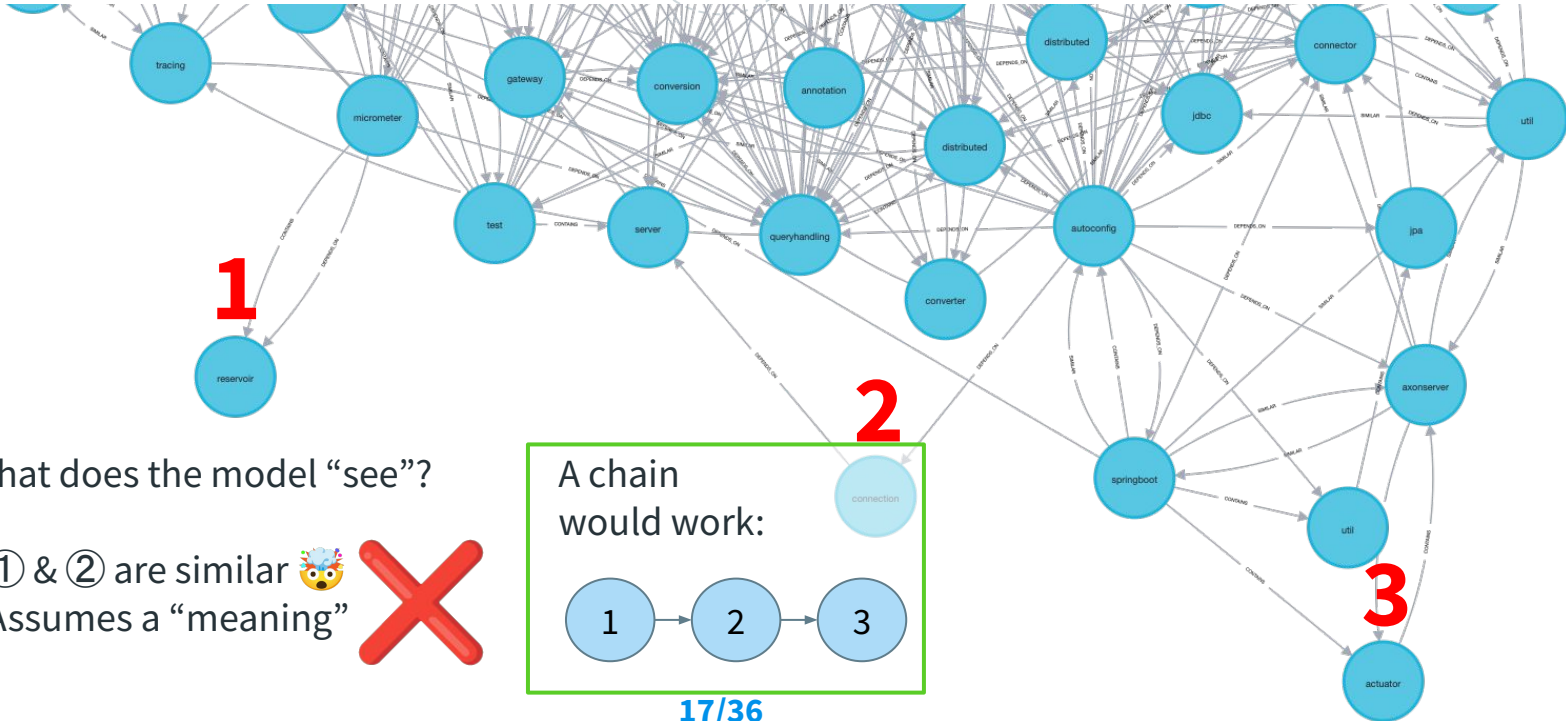
?

How can we represent
our **graph**
using **numbers**?

2) Machine Learning Preparation - Incrementing Identifiers?



2) Machine Learning Preparation - Incrementing Identifiers?



What does the model “see”?

- ① & ② are similar 🤖
- Assumes a “meaning”



2) Machine Learning Preparation



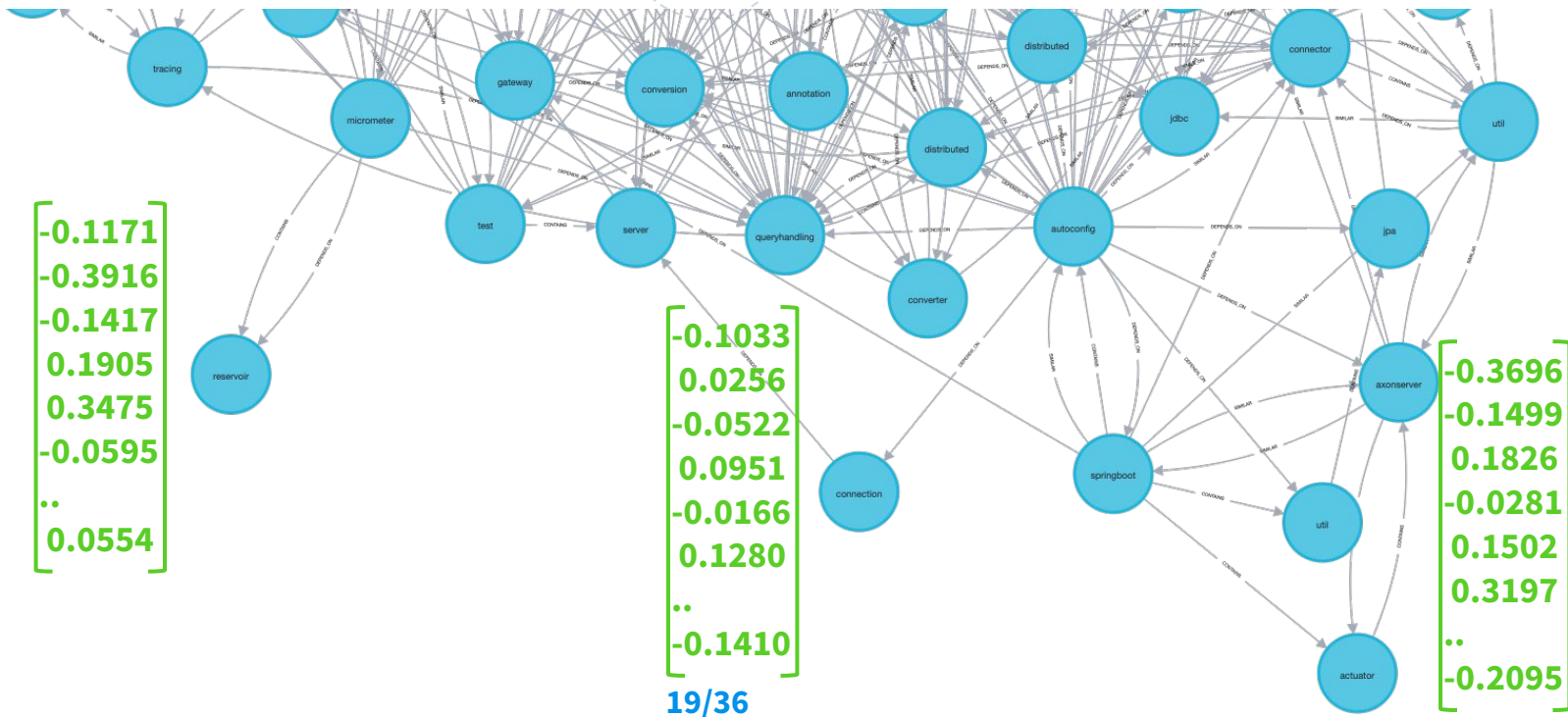
What do we actually want?

Highly connected code units have a **similar** representation.

Related code parts form **clusters**.

Unusual or not clearly assigned code units are **easy to isolate**.

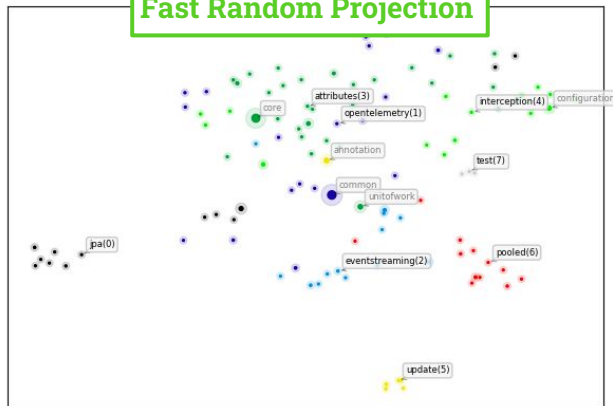
2) Machine Learning Preparation - Graph Node Embeddings



2) Machine Learning Preparation - Graph Node Embeddings

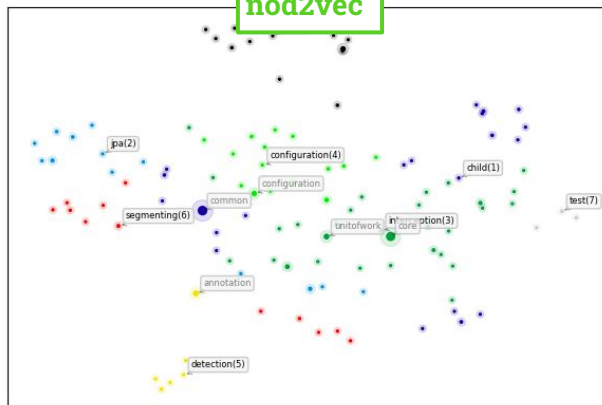
java Packages dependency graph node embeddings
Silhouette Score (aim higher)=0.1945, Davies-Bouldin Score (aim lower)=1.5736

Fast Random Projection



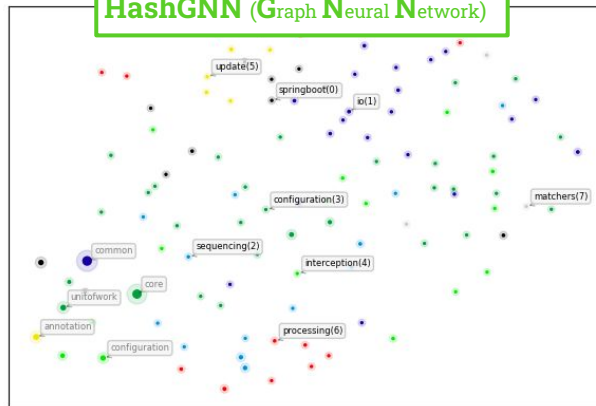
java Packages dependency graph node embeddings
Silhouette Score (aim higher)=0.2233, Davies-Bouldin Score (aim lower)=1.9820

nod2vec



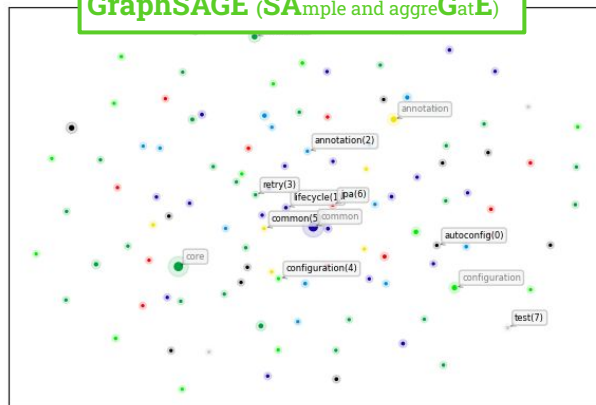
java Packages dependency graph node embeddings
Silhouette Score (aim higher)=-0.0232, Davies-Bouldin Score (aim lower)=3.6524

HashGNN (Graph Neural Network)



java Packages dependency graph node embeddings
Silhouette Score (aim higher)=-0.0850, Davies-Bouldin Score (aim lower)=0.5344

GraphSAGE (SAmples and aggreGatE)



Uniform
Manifold
Approximation and
Projection
for Dimension
Reduction



3.

Anomaly Detection

3) Anomaly Detection



How can we find
anomalies in code?
And **why**?

Find **unusual** code units
(e.g. Java Types)

Explorative
“surprise me”

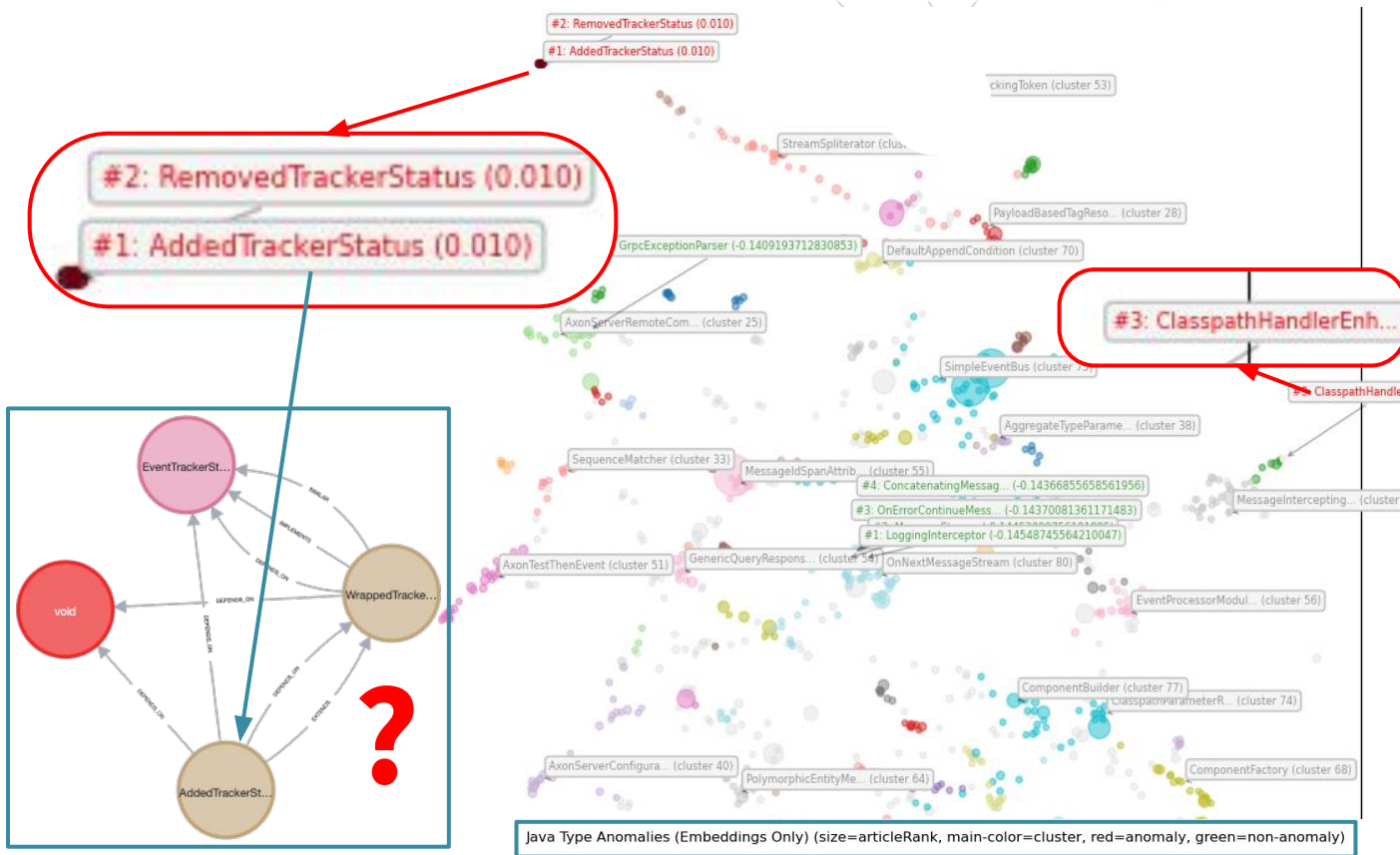
Without hand-written
rules

3) Anomaly Detection - Isolation Forest on Node Embeddings



Let's try **Isolation Forest** on
Graph Node Embeddings...

3) Anomaly Detection - Isolation Forest on Node Embeddings



3) Anomaly Detection - Feature Engineering



What are **good** features?

Complement each other

- local vs. global
- structural vs. semantic
- contextual

Well-balanced

Balance feature influence by reducing embedding dimensionality.

Standardized

Standardize feature values so they are **in the same range**.

3) Anomaly Detection - Feature Engineering

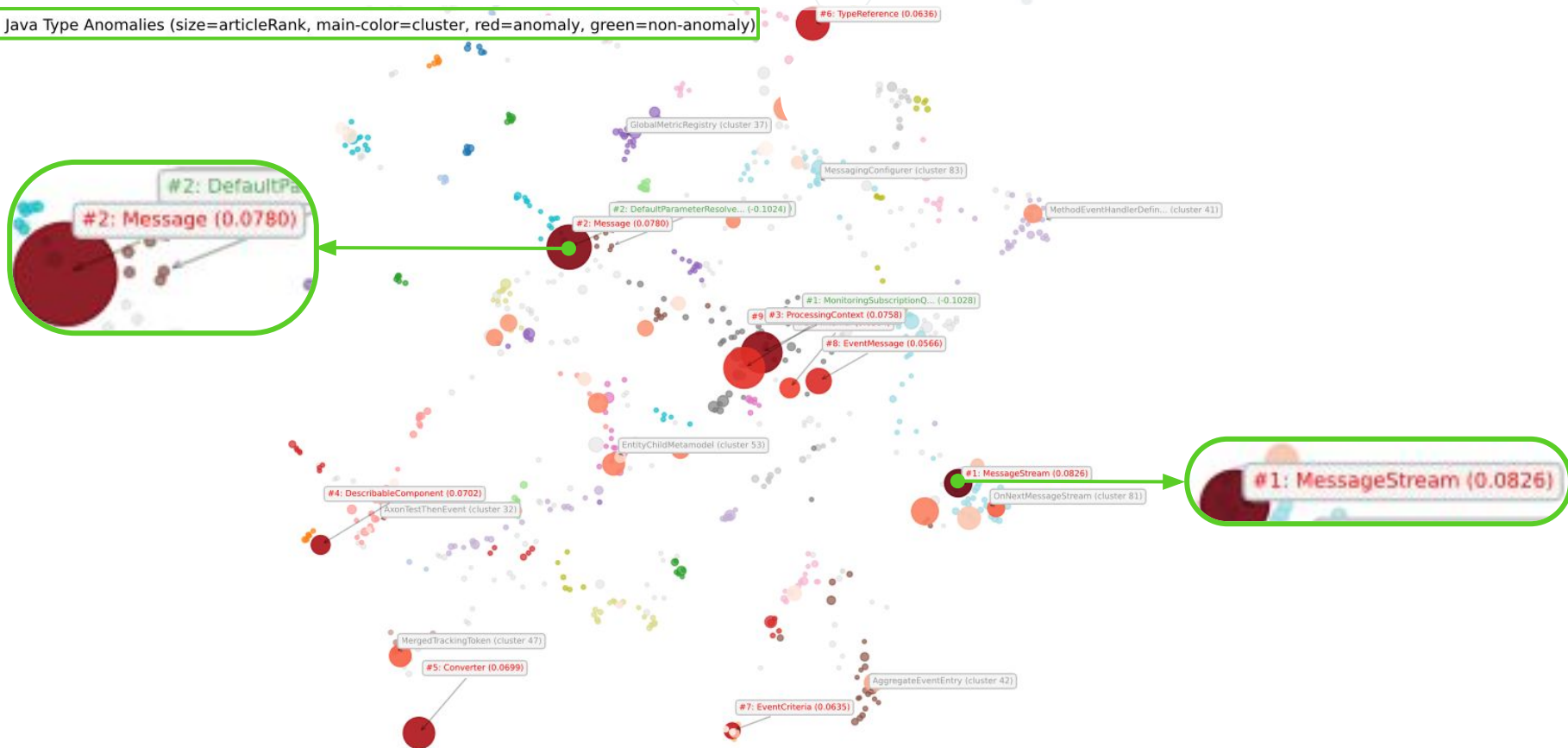


Examples

- **Degree (in / total)** → raw connectivity signal (local dependencies)
- **PageRank** → “How much depends on this?” (global importance)
- **PageRank – ArticleRank** → Architectural role mismatches (global structure)
- **Abstractness** → ratio of interfaces and abstract classes to all types (semantic)
- ...

3) Anomaly Detection - Feature Engineering

Java Type Anomalies (size=articleRank, main-color=cluster, red=anomaly, green=non-anomaly)

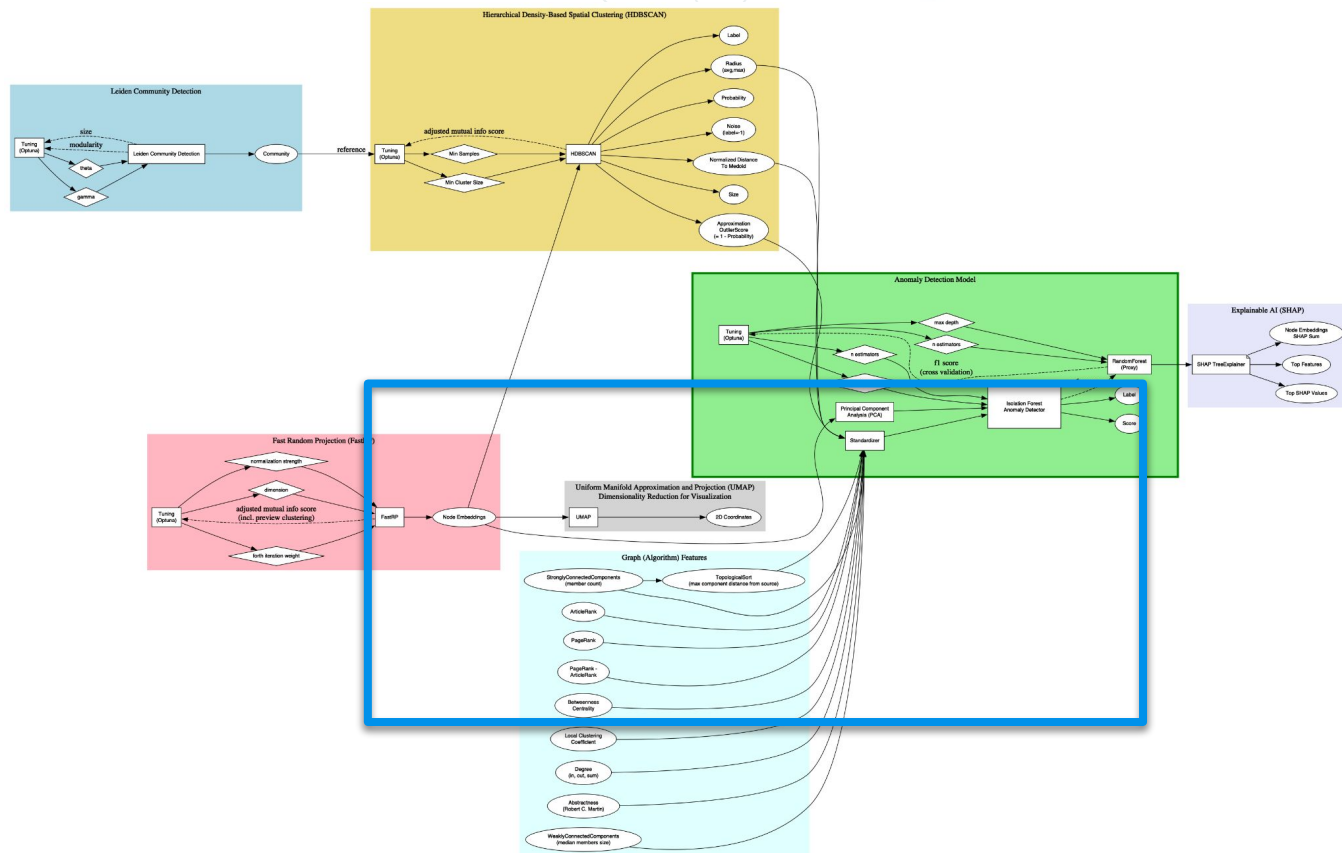


A decorative network diagram in the top-left corner, featuring a complex web of interconnected nodes and lines, with some nodes highlighted in blue and others in grey.

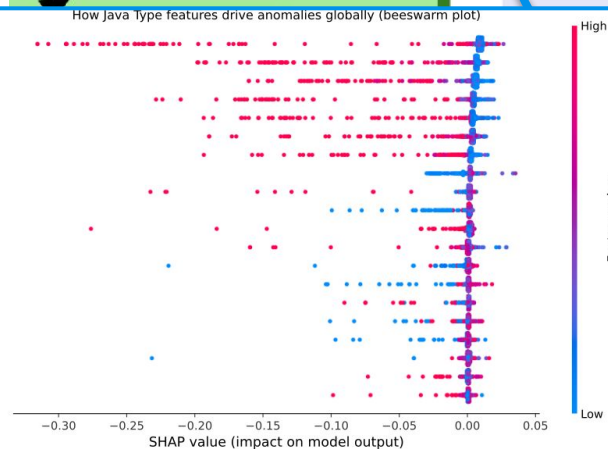
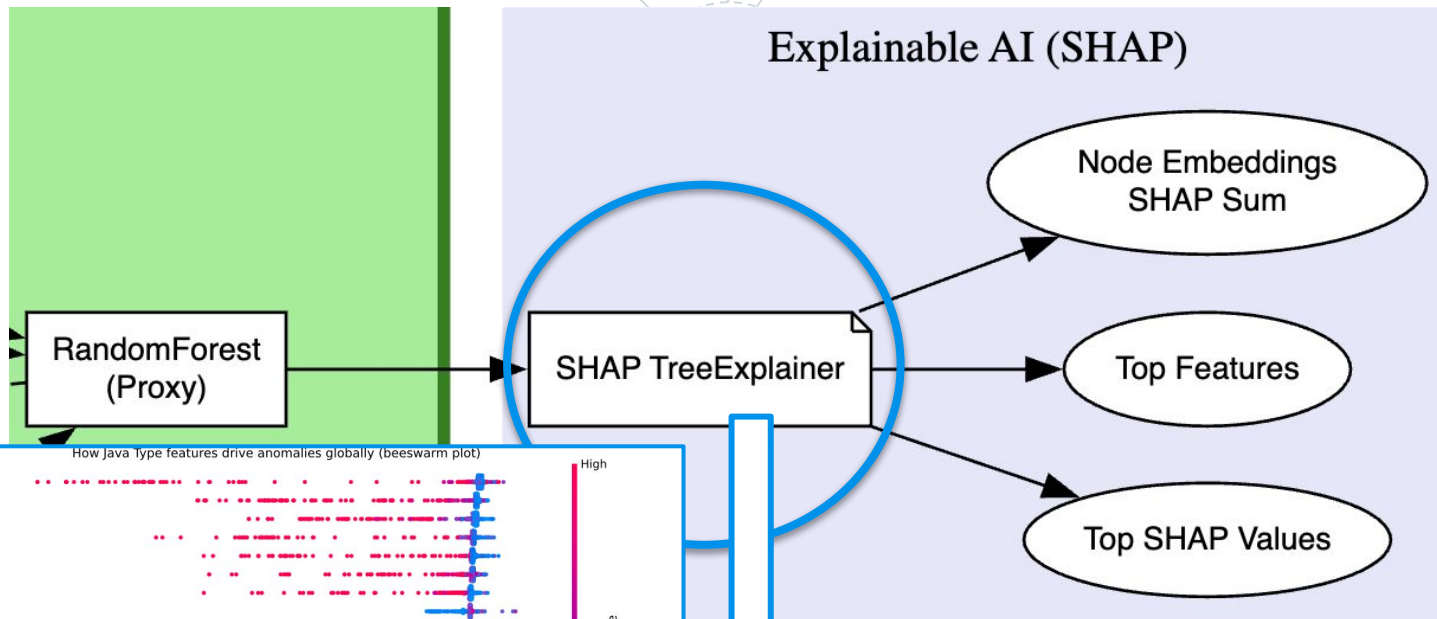
Outlook

What else?

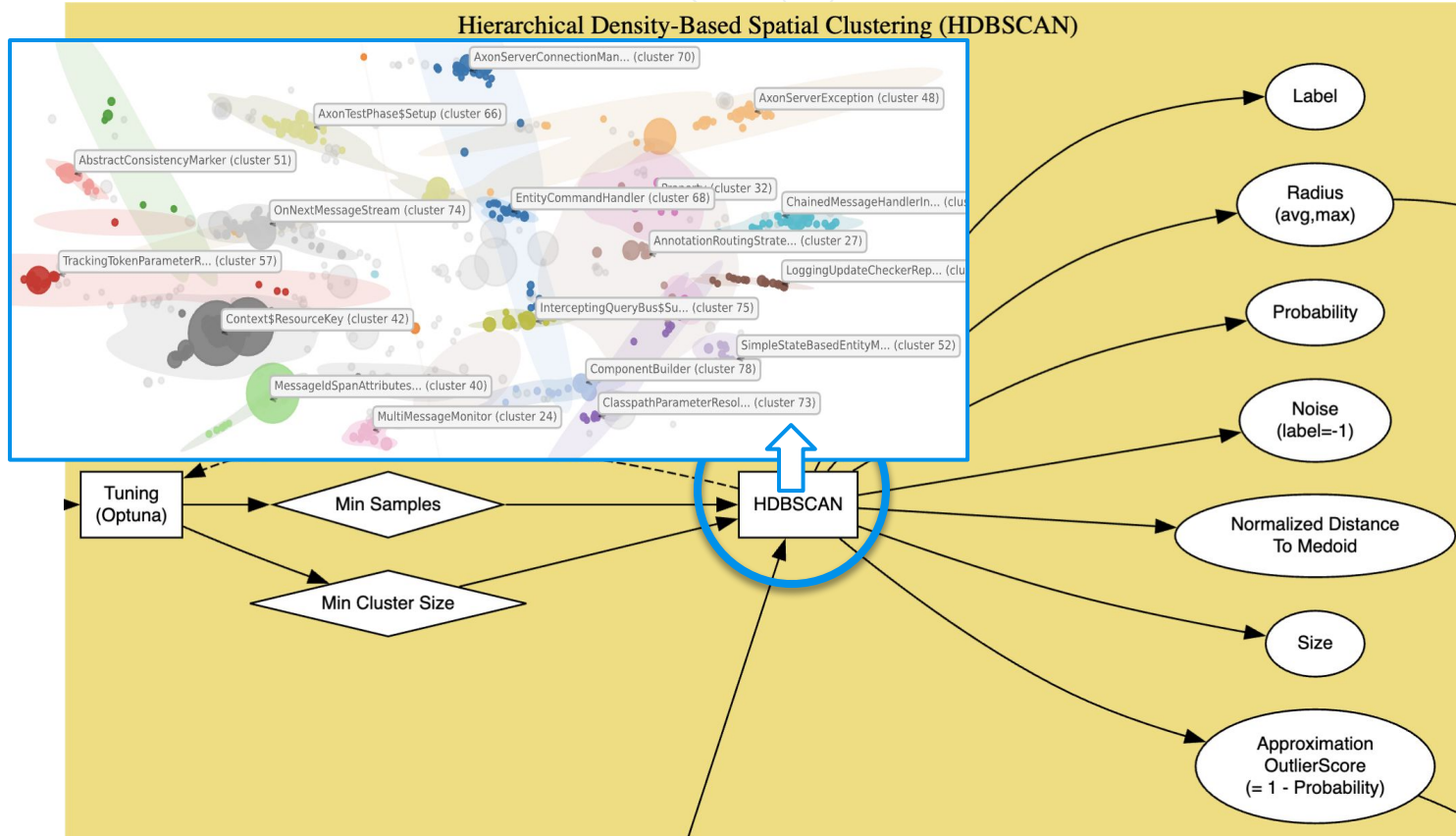
Outlook - Full Anomaly Detection Pipeline



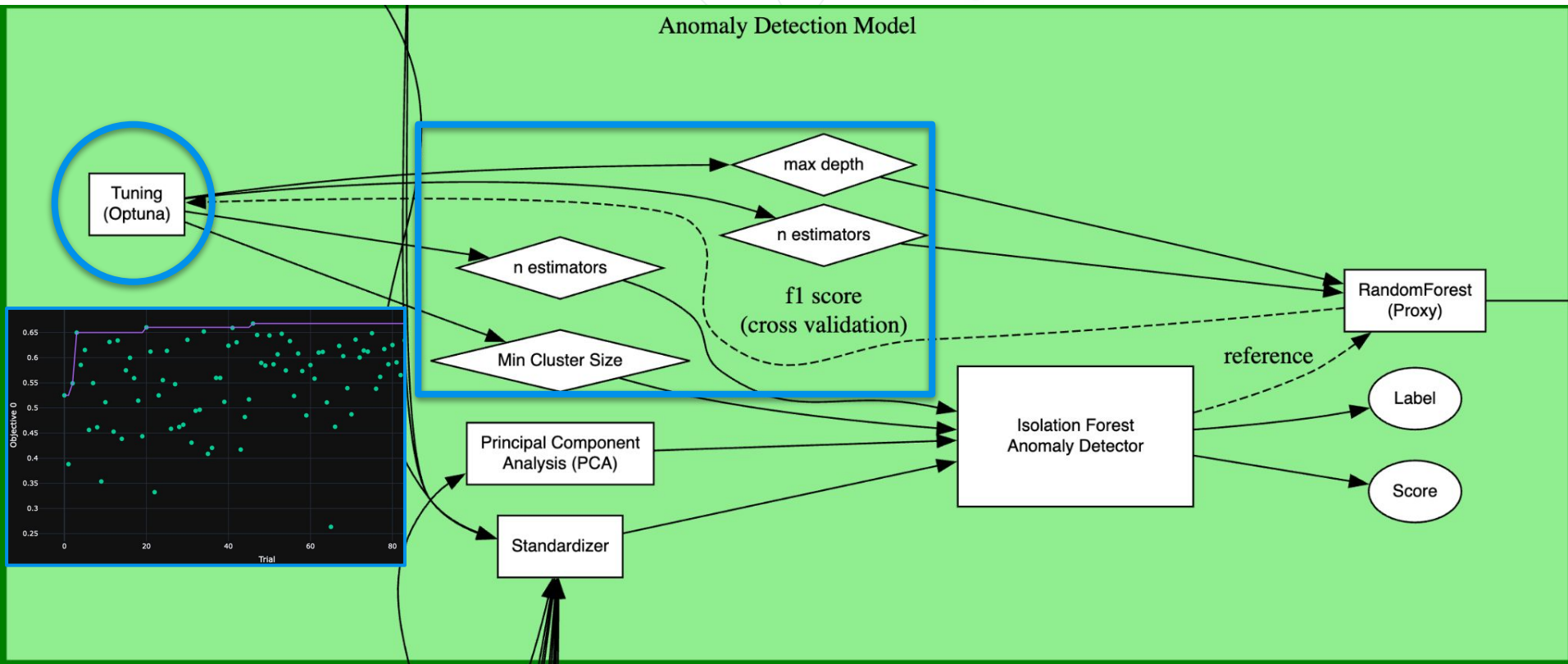
Outlook - Full Pipeline Architecture - Explainability



Outlook - Full Pipeline Architecture - Clustering



Outlook - Full Pipeline Architecture - Tuning



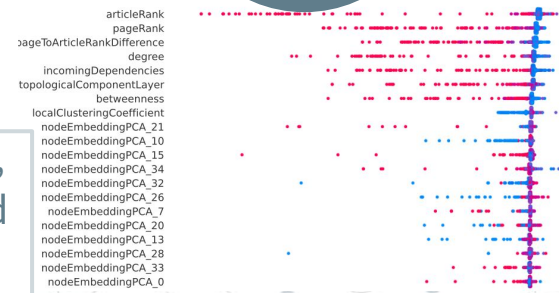
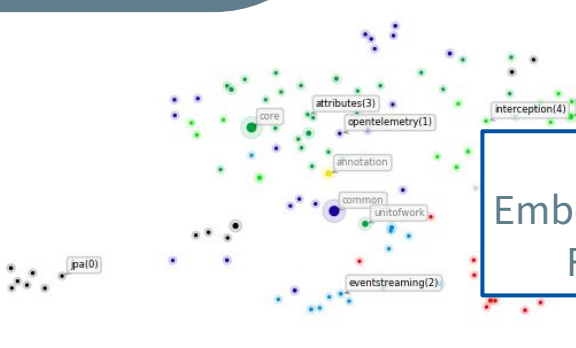
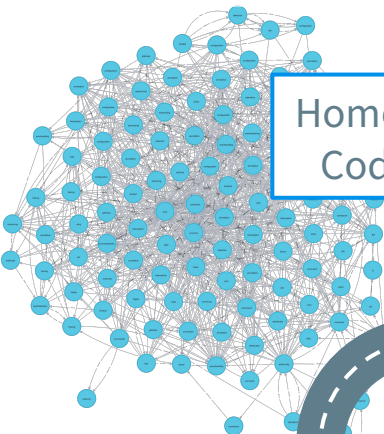
Wrap-up

Homogeneous
Code Graph

Anomaly
Detection

Node
Embeddings and
Features

Explainability,
Clustering and
Tuning



Outlook: Want to dig deeper or try it out **yourself?**



The README
includes a link
to the slides.

<https://github.com/JohT/code-graph-analysis-pipeline>

A decorative network diagram in the top-left corner, featuring a complex web of interconnected nodes and lines. The nodes are represented by small circles, some of which are larger and have concentric circles, suggesting different levels of connectivity or importance. The lines are thin and gray, creating a mesh-like structure.

Demo

References

code-graph-analysis-pipeline (JohT):
<https://github.com/JohT/code-graph-analysis-pipeline>

jQAssistant Software Analysis:
<https://github.com/jqassistant>

Neo4j Graph Database:
<https://neo4j.com>

Object-Oriented **Design Metrics**:
<https://kariera.future-processing.pl/blog/object-oriented-metrics-by-robert-martin>

Node Embeddings (Neo4j Graph Data Science):
<https://neo4j.com/docs/graph-data-science/current/machine-learning/node-embeddings>

Isolation Forest (scikit-learn):
<https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.IsolationForest.html>

SHAP (SHapley Additive exPlanations):
<https://shap.readthedocs.io/en/latest/index.html>

Optuna Hyperparameter Optimization (library):
<https://optuna.org>

Plotly Visualization (library):
<https://plotly.com/python/treemaps>

Anomaly Detection Pipeline Architecture:
<https://github.com/JohT/code-graph-analysis-pipeline/blob/main/domains/anomaly-detection/documentation/Architecture.svg>

Anomaly Detection Results (example):
<https://github.com/JohT/code-graph-analysis-examples?tab=readme-ov-file#java-type-anomalies>

Machine Learning with Graphs (Stanford)
https://youtu.be/JAB_plj2rbA?si=KcQSHHNVfs4e03N0